

Guía personal para el demo

Jupyter Notebooks en VS Code con Codex

Propósito: recuperar rápidamente el flujo de trabajo de Jupyter dentro de Visual Studio Code y ejecutar con seguridad el demo de pérdida silenciosa de observaciones.

Diseñado para: Kristian López Vargas Entorno principal: macOS, Visual Studio Code, Python, Jupyter y la extensión oficial de Codex Actualización: agosto de 2026

1. La idea del entorno

Durante el demo, VS Code será el laboratorio visible. En una sola ventana tendrás:

- el explorador de archivos a la izquierda;
- el notebook en el centro;
- Codex en un panel lateral a la derecha;
- la salida de cada celda debajo del código;
- la terminal integrada disponible, pero cerrada mientras no se necesite.

No necesitas iniciar un servidor Jupyter manualmente. La extensión de Jupyter puede ejecutar el notebook usando un entorno local de Python que contenga `ipykernel`.

El principio operativo es:

Codex ayuda a leer, diagnosticar y proponer. El kernel ejecuta. Tú revisas el cambio, interpretas el resultado y decides.

2. Preparación mínima, una sola vez

2.1 Instala o confirma estas aplicaciones

- 1 Visual Studio Code.
- 2 Python 3.11 o posterior. Una versión reciente y estable es suficiente.
- 3 Una cuenta de ChatGPT compatible con Codex.

Para comprobar Python en macOS, abre Terminal y ejecuta:

```
python3 --version
```

Si el comando no existe, instala Python antes de continuar. No uses el Python interno de macOS como entorno del curso.

2.2 Instala tres extensiones en VS Code

Abre la vista **Extensions** con `Cmd+Shift+X` en macOS o `Ctrl+Shift+X` en Windows/Linux. Busca e instala:

- 1 **Python**, publicada por Microsoft (`ms-python.python`).
- 2 **Jupyter**, publicada por Microsoft (`ms-toolsai.jupyter`).
- 3 **Codex** - OpenAI's coding agent, publicada por OpenAI (`openai.chatgpt`).

Comprueba siempre el **nombre del publicador**. Evita extensiones comunitarias que usan “ChatGPT” o “Codex” en el nombre pero no son la extensión oficial.

2.3 Abre Codex y autentícate

en el compositor de **Codex**.

- 1 Selecciona el icono de Codex en la barra lateral de VS Code.
- 2 Si no aparece, abre la Command Palette con `Cmd+Shift+P` y ejecuta `Codex: Open Codex Sidebar`.
- 3 Inicia sesión con tu cuenta de ChatGPT.
- 4 Para este demo, trabaja en modo Local; si hace falta, ~~escribe /local~~.

Codex puede usar como **contexto los archivos abiertos y las selecciones** del editor. También **puedes seleccionar código y ejecutar Codex**: `Add to Thread`, o añadir el archivo completo al chat desde la Command Palette.

3. Prepara dos espacios: maestro y demo limpio

La **carpeta maestra** incluida en el repositorio tiene esta estructura real:

```
data_audit_demo/
├── data/
│   ├── estudiantes.csv
│   └── características_hogar.csv
├── notebooks/
│   ├── 01_demo_inicial.ipynb
│   └── 02_demo_resuelto.ipynb
├── expected/
│   ├── metadata.json
│   └── resultados_esperados.md
├── scripts/
│   ├── execute_notebooks.py
│   ├── generate_demo.py
│   └── validate_demo.py
├── requirements.txt
└── README.md
```

Esta carpeta **maestra sirve para preparar, validar y recuperar el demo**. No la abras como carpeta de trabajo durante la demostración: Codex puede leer el notebook resuelto, los resultados esperados y el generador aunque no estén abiertos en el editor.

3.1 Crea la carpeta limpia para la clase

En macOS o Linux, desde data_audit_demo/:

```
mkdir -p ~/Desktop/data-audit-live/data
cp notebooks/01_demo_inicial.ipynb ~/Desktop/data-audit-live/
cp data/*.csv ~/Desktop/data-audit-live/data/
cp requirements.txt ~/Desktop/data-audit-live/
```

En Windows PowerShell, desde data_audit_demo/:

```
New-Item -ItemType Directory -Force "$HOME\Desktop\data-audit-live\data"
Copy-Item notebooks\01_demo_inicial.ipynb "$HOME\Desktop\data-audit-live\"
Copy-Item data\*.csv "$HOME\Desktop\data-audit-live\data\"
Copy-Item requirements.txt "$HOME\Desktop\data-audit-live\"
```

La carpeta que abrirás en vivo debe contener únicamente:

```
data-audit-live/
├── data/
│   ├── estudiantes.csv
│   └── características_hogar.csv
├── 01_demo_inicial.ipynb
└── requirements.txt
```

No copies 02_demo_resuelto.ipynb, expected/ ni scripts/. Esta separación evita que el agente revele los números o el mecanismo antes de que la sala los descubra.

3.2 Abre la carpeta, no solamente el notebook

- 1 En VS Code, selecciona File > Open Folder.
- 2 Elige data-audit-live.
- 3 Si VS Code pregunta por Workspace Trust, confía únicamente porque la carpeta fue creada y revisada por ti.

Abrir la carpeta limpia completa permite que las rutas relativas encuentren data/ y limita el contexto de Codex a los archivos permitidos. La carpeta maestra permanece cerrada, pero disponible como respaldo fuera de la ventana proyectada.

4. Crea un entorno de Python aislado

Esta es la ruta más estable para el demo. Abre la terminal integrada con Terminal > New Terminal.

macOS o Linux

```
cd ~/Desktop/data-audit-live
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Windows PowerShell

```
cd $HOME\Desktop\data-audit-live
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

El archivo de **requisitos** instala:

- **pandas** para abrir, unir y resumir datos;
- **matplotlib** para cualquier gráfico breve;
- **ipykernel** para ejecutar Python como kernel del notebook;
- nbformat y nbclient para validar ejecuciones limpias fuera de VS Code.

No es necesario instalar el paquete completo jupyter para que VS Code ejecute un notebook Python local.

5. Selecciona intérprete y kernel

El intérprete de Python y el kernel del notebook están relacionados, pero VS Code puede mostrarlos por rutas distintas. Verifica ambos.

5.1 Selecciona el intérprete del proyecto

- 1 Abre la Command Palette.
- 2 Ejecuta Python: Select Interpreter.
- 3 Elige el Python ubicado dentro de .venv.

En macOS normalmente termina en:

```
.venv/bin/python
```

En Windows normalmente termina en:

```
.venv\Scripts\python.exe
```

5.2 Selecciona el kernel del notebook

- 1 Abre 01_demo_inicial.ipynb en la carpeta limpia.
- 2 En la esquina superior derecha, selecciona Select Kernel.

- 3 Si aparece `.venv` entre los recientes, selecciónalo.
- 4 Si no aparece, elige `Select Another Kernel > Python Environments` y selecciona `.venv`.

VS Code recuerda normalmente el último kernel usado por el notebook.

5.3 Ejecuta una prueba de identidad

En una celda nueva, ejecuta:

```
import sys
from pathlib import Path

print("Python:", sys.executable)
print("Carpeta de trabajo:", Path.cwd())
```

La primera línea debe apuntar a `.venv`. La carpeta de trabajo debe permitir que las rutas relativas del notebook encuentren `data/`.

El **notebook entregado busca data/** desde la carpeta abierta. No cambies el directorio de trabajo manualmente durante el demo.

6. Operaciones básicas del notebook

Ejecutar celdas

- Botón triangular a la izquierda: ejecuta una celda.
- `Ctrl+Enter`: ejecuta la celda seleccionada.
- `Shift+Enter`: ejecuta la celda y avanza a la siguiente.
- `Run All`: ejecuta todas las celdas en orden.

En macOS, los atajos de ejecución de celdas también usan `Ctrl`, no `Cmd`.

Celdas de código y Markdown

Usa celdas Markdown para:

- **declarar la pregunta;**
- explicar la población analítica;
- anticipar qué debería ocurrir;
- interpretar resultados;
- registrar decisiones.

Usa celdas de **código para una sola operación conceptual por celda**. Para el demo, evita celdas largas que cargan, limpian, unen y resumen todo de una vez.

Reiniciar el kernel

El **kernel conserva variables entre ejecuciones**. Por eso un notebook puede funcionar después de ejecutar celdas fuera de orden y fallar al abrirlo desde cero.

Antes de la clase, usa la barra superior del notebook para:

- 1 Restart Kernel.
- 2 Run All.
- 3 Confirmar que termina sin depender de memoria previa.

Si una celda queda ejecutándose indefinidamente, usa `Interrupt Kernel`. Si el estado queda confuso, reinicia el kernel.

Variables y datos

Después de ejecutar celdas, **abre Variables desde la barra del notebook**. Puedes inspeccionar data frames y abrirlos en el Data Viewer. Esto es útil para verificar columnas, tipos y tamaño sin llenar la presentación de impresiones extensas.

7. Configuración visual para enseñar

Antes del demo:

- 1 Mueve el panel de Codex a la derecha.
- 2 Deja Explorer a la izquierda.
- 3 Usa zoom o una fuente suficientemente grande para proyección.
- 4 Cierra paneles no utilizados, notificaciones y archivos irrelevantes.
- 5 Deja visible el nombre del kernel.
- 6 Colapsa celdas auxiliares que no forman parte del relato.
- 7 Mantén la terminal cerrada hasta necesitarla.

Una composición recomendable es:

Explorer	Notebook	Codex
data/ data/*.csv	Pregunta Código Resultado	Prompt Diagnóstico Cambio propuesto

No intentes mostrar simultáneamente notebook, terminal, Explorer y Codex con el mismo peso. El notebook es el centro; Codex aparece cuando dialogas con él; la terminal aparece solo para verificar o instalar.

8. Flujo seguro con Codex

Los siguientes son los cinco prompts del slideshow final. Pégalos en orden. Aunque la carpeta limpia no contiene la solución, cada prompt repite el límite para que el contrato quede visible y sobreviva si cambia el contexto del chat.

Prompt 1: comprender antes de editar

Abre el notebook y envía:

```
Lee únicamente 01_demo_inicial.ipynb y los datos que usa.
No consultes 02_demo_resuelto.ipynb, expected/ ni
scripts/generate_demo.py.
Sin modificar archivos, resume la pregunta, la población
y cada transformación que pueda cambiar el número o la
composición de las observaciones. Señala incertidumbres.
```

Resultado mínimo esperado: Codex debe identificar el merge como una operación capaz de cambiar el número o la composición de las filas. No autorices cambios.

Prompt 2: diagnosticar antes de corregir

No corrijas todavía el notebook. Propón celdas diagnósticas para localizar dónde se pierden observaciones: conteos antes y después, un indicador de merge y tasas de no correspondencia por Juntos, ruralidad y distrito.
No consultes 02_demo_resuelto.ipynb, expected/ ni scripts/generate_demo.py.

Entre prompts: revisar y ejecutar diagnósticos

Puedes copiar las celdas propuestas o pedir a Codex que las inserte. Si edita el notebook:

- 1 revisa el resumen;
- 2 inspecciona el diff;
- 3 acepta solamente las celdas que comprendes;
- 4 ejecuta tú las celdas;
- 5 interpreta el resultado antes del siguiente prompt.

Los números que debes obtener son 716 estudiantes antes del cruce, 561 después y 155 perdidos. **La correspondencia es 91.9 % en zona urbana y 68.2 % en zona rural. No los anuncies antes de ejecutar los diagnósticos.**

Prompt 3: pedir la corrección mínima

Ya confirmamos que el inner merge elimina observaciones de manera desigual. Propón el cambio mínimo que conserve todos los estudiantes durante la auditoría: el tipo de cruce, un indicador de correspondencia y una validación de la cardinalidad esperada. No alteres todavía la definición final de la muestra. No consultes 02_demo_resuelto.ipynb, expected/ ni scripts/generate_demo.py.

La propuesta esperada usa `how='left'`, `indicator=True` y `validate='many_to_one'`. Si Codex omite el indicador o la validación, añádelos a mano y dilo en voz alta: el agente propone; el criterio lo ponemos nosotros.

Prompt 4: convertir el hallazgo en comprobaciones

Añade tres comprobaciones explícitas: una debe detectar cambios no autorizados en el número de estudiantes; otra debe fallar si `estudiante_id` deja de ser único; la tercera debe fallar si la tasa de correspondencia del merge baja del umbral definido. Explica por qué elegiste cada comprobación. No consultes 02_demo_resuelto.ipynb, expected/ ni scripts/generate_demo.py.

Antes de las aserciones, crea la variable que mide correspondencia:

```
auditoria['en_tabla_auxiliar'] = (
    auditoria['_merge'].eq('both')
)
```

Prueba que la última aserción puede fallar: ejecuta primero el umbral 0.70 y después cámbialo temporalmente a 0.80.

Prompt 5: separar descripción de causalidad

Compara la tabla inicial y la tabla auditada. Separa claramente:

- 1) lo que estos descriptivos muestran;
- 2) explicaciones alternativas;
- 3) lo que no puede interpretarse causalmente.

No redactes una conclusión causal. No consultes 02_demo_resuelto.ipynb, expected/ ni scripts/generate_demo.py.

El descriptivo correcto usa los 716 estudiantes; la tabla auxiliar se reporta como auditoría de cobertura, no como requisito para definir la muestra. La brecha descriptiva cambia de 0.058 en la submuestra del `inner merge` a 0.077 en la población analítica declarada. Esto no identifica un efecto de Juntos.

9. Runbook del demo, minuto a minuto

Antes de iniciar

- Valida la carpeta maestra con `python scripts/validate_demo.py`.
- Reinicia y ejecuta ambos notebooks desde un kernel limpio.
- Cierra la carpeta maestra.
- Abre únicamente `data-audit-live`.
- Activa `.venv`.
- Selecciona el kernel correcto.
- Abre Codex en un chat nuevo.
- Cierra la terminal.
- Confirma que la carpeta abierta no contiene `02_demo_resuelto.ipynb`, `expected/` ni `scripts/`.
- Ten el slideshow y el notebook inicial preejecutado como respaldo.

Minuto 0:00-1:10 - encuadrar y presentar la pregunta

- 1 Abre con: "Este notebook no tiene errores. Ese es el problema".
- 2 Nombra Explorer, notebook y Codex sin enseñar instalación.
- 3 Lee la pregunta y subraya que es descriptiva.

Minuto 1:10-2:20 - mostrar el resultado plausible

- 1 Ejecuta las celdas de carga y la tabla inicial.
- 2 Pregunta: "¿Qué comprobarían antes de aceptar esta comparación?"
- 3 Sostén treinta segundos de silencio antes de intervenir.
- 4 Si nadie responde, pregunta cuántos estudiantes hay en la tabla.

Minuto 2:20-3:55 - comprender antes de editar

- 1 Envía el prompt 1.
- 2 Busca la mención del `merge`.
- 3 Muestra la línea `how='inner'`.
- 4 No aceptes todavía una corrección.

Minuto 3:55-6:50 - diagnosticar y localizar la pérdida

- 1 Envía el prompt 2.
- 2 Ejecuta conteos antes y después: 716 a 561.
- 3 Muestra que desaparecen 155 estudiantes, 21.6 %.
- 4 Compara la cobertura urbana, 91.9 %, con la rural, 68.2 %.
- 5 Verbaliza: "El código corre, pero cambió quién está en los datos".

Minuto 6:50-9:45 - corregir y proteger

- 1 Envía el prompt 3 y revisa el diff antes de aceptarlo.
- 2 Ejecuta el `left merge` con indicador y validación de cardinalidad.
- 3 Envía el prompt 4 y deja las tres aserciones en el notebook.
- 4 Cambia 0.70 a 0.80 para demostrar que una comprobación puede fallar.
- 5 Explica que, para este descriptivo, el cruce auxiliar no era necesario.

Minuto 9:45-12:30 - interpretar y cerrar

- 1 Envía el prompt 5.
- 2 Compara 0.058 con 0.077 y nombra el cambio de población.
- 3 Explica por qué la corrección técnica no identifica causalidad.
- 4 Resume los papeles: Codex propone, el kernel ejecuta, la persona decide.
- 5 Termina en la lámina de materiales.

Minuto 12:30-17:30 - transferencia opcional

Pide una nota de cuatro líneas sobre una transformación propia: población, conteo antes/después, quién entra o sale y comprobación que quedará escrita. La primera pregunta es si el merge, filtro, dropna o deduplicación hacía falta para responder la pregunta.

10. Qué debe estar preparado en las láminas

No uses las capturas como tutorial de botones. Prepara seis estados:

- 1 La pregunta y los dos archivos.
- 2 La tabla inicial aparentemente convincente.
- 3 La línea del `inner merge` resaltada.
- 4 El conteo antes/después y la composición de los registros perdidos.
- 5 El diff de la corrección y las tres comprobaciones.
- 6 La tabla auditada y el límite inferencial.

Estas imágenes son también el fallback si internet, Codex o el kernel fallan durante la clase.

11. Problemas frecuentes

“Select Kernel” no muestra `.venv`

- 1 Confirma que `.venv` existe dentro de la carpeta abierta.
- 2 Ejecuta en la terminal:

```
.venv/bin/python -m pip install ipykernel
```

En Windows usa `.venv\Scripts\python.exe`.

- 1 Ejecuta Developer: Reload Window.
- 2 Vuelve a Select Kernel > Select Another Kernel > Python Environments.

Error: ModuleNotFoundError

El paquete fue instalado en otro Python. Ejecuta la celda con `sys.executable`, abre una terminal y usa exactamente ese intérprete:

```
python -m pip install NOMBRE_DEL_PAQUETE
```

Evita usar `pip` solo; `python -m pip` reduce el riesgo de instalar en otro entorno.

El notebook funciona solo cuando ejecutas celdas manualmente

Hay estado oculto. Reinicia el kernel y ejecuta Run All. Reordena celdas hasta que el notebook funcione de arriba hacia abajo.

Las rutas a `data/` fallan

Confirma `Path.cwd()`. Abre la carpeta raíz del proyecto y usa rutas construidas desde una raíz explícita. No dependas de cambios manuales de directorio durante el demo.

Codex no usa el notebook correcto

- 1 Mantén abierto el notebook.
- 2 Selecciona las celdas relevantes y usa Add to Thread.
- 3 Nombra `01_demo_inicial.ipynb` y los CSV permitidos en el prompt.
- 4 No adjuntes resultados esperados ni el notebook resuelto.
- 5 Si el contexto parece contaminado, abre un chat nuevo y repite el límite de archivos.

El kernel queda ocupado

Usa Interrupt Kernel. Si no responde, usa Restart Kernel y ejecuta de nuevo desde arriba.

VS Code abre la carpeta en Restricted Mode

No ejecutes código de una carpeta desconocida. Para tu carpeta revisada, abre el indicador de Workspace Trust y marca el espacio como confiable.

Codex propone demasiados cambios

Cancela o rechaza el diff y reduce el alcance del prompt: "Añade solo una celda diagnóstica; no refactorices ni cambies otras celdas."

12. Ensayo y fallback

Un día antes

- Actualiza VS Code y las tres extensiones, y después congela el entorno.
- Reinstala dependencias desde `requirements.txt` en una `.venv` limpia.
- Ejecuta Restart Kernel + Run All.
- Revisa que el notebook inicial produzca el resultado pedagógico esperado.
- Revisa que el notebook resuelto termine correctamente.
- Regenera `data-audit-live` y confirma que no contiene la solución.
- Graba una ejecución corta o toma las seis capturas.
- Guarda una copia local; no dependas de sincronización en la nube.

Quince minutos antes

- Desactiva notificaciones.
- Conecta el cargador.
- Comprueba proyección y tamaño de fuente.
- Abre `data-audit-live`, el notebook, Codex y la presentación.
- Ejecuta una celda sencilla para comprobar el kernel.
- Envía un prompt corto a Codex para comprobar la sesión.
- Vuelve al estado inicial del demo.

Si Codex falla

Continúa con las respuestas preparadas y pregunta al grupo qué diagnóstico pediría. Muestra las salidas de las láminas o abre el notebook resuelto fuera de la carpeta proyectada, sin incorporarlo al chat.

Si el kernel falla

Usa las capturas de resultados y explica el razonamiento. No gastes tiempo de clase reinstalando Python.

Si el tiempo se reduce

Muestra el diagnóstico ya preparado, ejecuta solo una aserción y conserva el cierre inferencial.

13. Lista corta para imprimir

Preparación

- ☐ VS Code actualizado.
- ☐ Extensiones oficiales Python, Jupyter y Codex activas.
- ☐ Carpeta limpia `data-audit-live` abierta.
- ☐ Notebook resuelto, `expected/` y `scripts/` ausentes de esa carpeta.
- ☐ `.venv` creada y dependencias instaladas.
- ☐ Intérprete y kernel apuntan a `.venv`.
- ☐ Notebook inicial y resuelto probados desde kernel limpio.
- ☐ Codex autenticado y funcionando en modo local.
- ☐ Capturas o video de respaldo disponibles.

Durante el demo

- ☐ Explicar antes de editar.
 - ☐ Diagnosticar antes de corregir.
 - ☐ Contar observaciones antes y después.
 - ☐ Revisar quién desaparece.
 - ☐ Inspeccionar el diff.
 - ☐ Ejecutar la corrección.
 - ☐ Añadir comprobaciones permanentes.
 - ☐ Separar resultado descriptivo de inferencia causal.
 - ☐ Preguntar si la transformación era necesaria para la pregunta.
-

Fuentes oficiales

- Visual Studio Code, Jupyter Notebooks in VS Code:
<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
- Visual Studio Code, Manage Jupyter Kernels:
<https://code.visualstudio.com/docs/datascience/jupyter-kernel-management>
- Visual Studio Code, Python environments: <https://code.visualstudio.com/docs/python/environments>
- Microsoft Marketplace, Jupyter extension: <https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter>
- OpenAI, Codex IDE extension: <https://developers.openai.com/codex/ide>
- OpenAI Marketplace listing, Codex - OpenAI's coding agent:
<https://marketplace.visualstudio.com/items?itemName=openai.chatgpt>

Las etiquetas exactas de algunos botones pueden variar ligeramente según la versión de VS Code o el sistema operativo. Las rutas principales verificadas son Command Palette, Python: Select Interpreter, Select Kernel y Codex: Open Codex Sidebar.